

PathfindingLib: A Pathfinding Library using SFML

Prem Gandhi¹

Abstract—Pathfinding behavior algorithms are crucial to enabling dynamic movement behavior that make gameplay exciting and noteworthy. Pathfinding algorithms are mostly implemented using two major graph search algorithms: Dijkstra's Algorithm and A*. The algorithms both use two lists called an "open list" and "closed list". This report implements both algorithms and displays a game implementation using Simple Fast Media Library (SFML). The approach utilizes various data structures to represent world data, specifically a map of the environment. The map contains obstacles and also contains connections between points that can be used to traverse the map.

I. INTRODUCTION

Pathfinding algorithms serve as an abstraction of movement behavior algorithms and work at the intersection between movement and decision-making. These algorithms must be fast and most, if not all, work on graph-based search approaches. There is an extensive body of literature on different graph-based, especially A* which uses a heuristic to determine the strategy for searching a graph.

Graph search algorithms are fundamentally split up into breadth-first search (BFS) and depth-first search (DFS) methods. Dijkstra's algorithm is fundamentally a BFS algorithm and it will always produce an optimal solution. The tradeoff for optimality is speed which is often more important to the game developer as marginal losses in optimality will not cause the player to lose their immersion in the game. Additionally, plans are adjusted quite frequently which additionally reduces the need for an optimal solution. Optimality can be reduced too much otherwise it can cause behaviors that may look odd or cause the player to lose the feeling of immersion. A* is a family of algorithms that includes Dijkstra's as a subset. A* is an algorithm that uses a heuristic to produce an estimated total cost which can help create a balance between BFS and DFS approaches. This allows for planning to be done much faster because domain knowledge can be used to discard nodes that will not lead to the goal node.

II. SOFTWARE ARCHITECTURE

The project was split up by three major folders: app, include, and src. Additionally, the boid image used in the sfml-demo is stored in the assets folder. The app folder contains two source files which are compiled into the main executables for the project: graph-analysis and sfml-demo.

^{*}This work was not supported by any organization

¹Prem Gandhi is with Industrial and Systems Engineering Masters Program, North Carolina State University, Raleigh, NC 27606 phgandh2@ncsu.edu

1) *graph-analysis*: The graph-analysis program compares the various algorithms and returns the speed and path planned by each on a randomly generated graph. The graph can be made larger or smaller according to preference. The implementation for the various pathfinding behaviors is located in the src folder which are split up by the pathfinding '.cpp' files and the movement behavior files. The code between the pathfinding and movement behavior files are completely separated.

The PathfindingLib implements a basic set of objects to hold and represent the graph structure.

- 1) Connection: This object holds the data about connections between nodes. It represents edges that compose a graph. The nodes can be acquired by searching all the connections and noting the set of 'fromNodes'. It is composed of a fromNode, toNode, and a cost to travel between the nodes.
- 2) Graph: This object represents a vector of the connections.
- 3) NodeRecord: This object is used to store the fromNode, connection, and cost so far for implementation in graph-based search algorithms. This object contains the information that is used to select the next node to search and determine optimality in paths.
- 4) PathfindingList: This object represents a vector of NodeRecords.

2) *sfml-demo*: The sfml-demo program (refer to Figure [9]) first creates an environment based on a "sketch" which is then converted into a map (e.g. array of arrays) that is converted into a graph (refer to Figure [11]). This enables rapid construction of new environments and the graph can automatically be generated based on the sketch.

The boid position is acquired along with the goal position which can be set via left mouse click. The nodes for the boid and goal position are calculated using a simple function and then passed to the path-finding algorithm. The localization function is shown in Figure [12] and the quantization function is shown in Figure [13]. Once the pathfinding algorithm is complete, it will return a path of nodes. The nodes are converted back to positions and then passed as way points to the boid. Once the boid reaches a way point, it will pop out the next way point from the vector containing the path. This behavior repeats until the goal node is reached.

III. EXPERIMENT

A. Pathfinding Speed

The various pathfinding algorithms were tested on randomly generated graphs and the algorithm that performed the

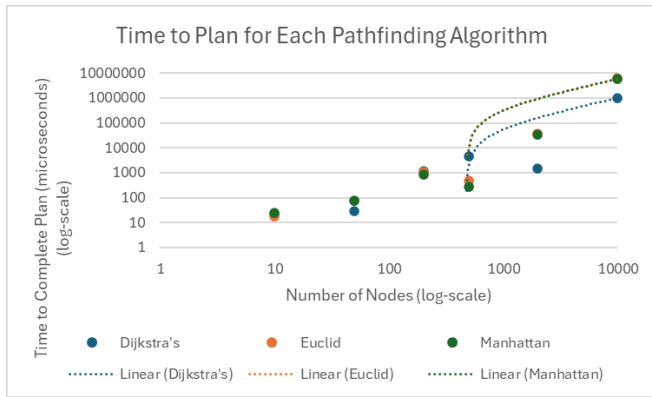


Fig. 1: Figure illustrates the time complexity of each algorithm based on the size of a random graph. Trendlines are included for each heuristic.

best was Dijkstra's at 231 microseconds 2. This baseline was compared with two A* algorithms using Euclidean distance and Manhattan Distance, respectively. The Euclidean heuristic retrieved the same path in 762 microseconds. Finally, the Manhattan heuristic retrieved the same path in 1081 seconds.

The various pathfinding algorithms were tested on the SFML map with the same start and end nodes of 42 and 1055, respectively. Dijkstra served as the baseline and completed pathfinding in 7747 microseconds. The euclidean heuristic was able to achieve a plan 1300 seconds while the manhattan heuristic completed pathfinding in 11395. Based on these results, the Euclidean heuristic was the most optimal as it calculated an estimated cost by turning the start and goal nodes into positions on the map and then calculated the distance. This contrasts significantly with the results above where the domain knowledge latent in the heuristic doesn't help the performance. Rather the performance is diminished. Dijkstra's performed around the middle and represents the fastest strategy for a guaranteed optimal plan. Finally, Manhattan seemed to do poorly on both random graph and SFML because the heuristic overestimates slightly.

In conclusion, Dijkstra's seems to be useful any general set of graph because it can handle random graphs with the most efficiency and performs well enough on the SFML map. Euclidean seems to offer the best kind of performance for lattice graphs like the SFML graph because the euclidean distance offers a cheap way to avoid checking nodes which are obviously not leading towards the goal.

B. Graph Construction and Size

The small graph was generated by using the example from AI for Games (3rd Ed.) by Ian Millington as a test example [Figure [14]]. This graph was chosen because it shows cyclical behavior and also has some nodes that are dead-ends. This ensures that the algorithm is actually working as it is supposed to. The graph is also simple enough to implement by hand and then test as done in the unit tests.

All the large graphs used in the assignment were generated programmatically. The first method implemented was a random graph generation which was chosen because of the ease

to create arbitrarily large graphs and perform pathfinding on them. The number of nodes and edges could be selected using parameters. The number of edges used remained at 3 for all the experimentation conducted. By setting the number of edges to 3 times the number of nodes, the pathfinding algorithms must use the find and contains functions frequently. This decision also ensures that the performance of the algorithms is less subject to the possibility of a series of single connection nodes from the start node to the end node.

The time complexity for each algorithm was measured and plotted in Figure [1]. This plot uses a vertical and horizontal log scale to fit the data points in a succinct chart. The figure illustrates that as the random graph grows larger, Dijkstra's seems to consistently perform better. Manhattan seems to consistently overestimate as an inadmissible heuristic while Euclidean distance is fairly similar in lower graph sizes until the graph grows extremely large. This means that it overestimates very slightly. A comparison of the plans for 10000 nodes showed that Manhattan had a much larger plan compared to Dijkstra. Euclidean heuristic seems to have a slightly larger plan; however, the construction of the plan took much longer.

C. Corridor Size and Map Construction

The sfml-demo program works well with a variety of different cell sizes. The program was evaluated with cell sizes: 10 [Figure [6]], 15 [Figure [7]], and 20 [Figure [8]]. The size that looked the best visually was cell size 15. One interesting discovery in evaluating the different sizes is the fact that the pathfinding algorithm works perfectly on each size. Sizes less than 10 were not tested as due to the visual limitations of differentiating between the boid and the corridor. A variety of maps can also be tested and the a maze like environment was chosen to implement the various cell size options. This map was selected because of the complex paths that would follow which emphasize the precision necessary for the quantization and localization methods used to do pathfinding.

The map could be easily modified as mentioned earlier by entering a map sketch similar to Figure [11]. The map was modified to test open area conditions and with the Euclidean heuristic. The results are shown in Figure [10] where the boid is able to successfully approach the point in the open area. When following the path, the boid performs a jagged movement motion because the nodes are only connected in a 4 way fashion because of the automatic grid generation function.

IV. CONCLUSIONS

In conclusion, this report demonstrates how pathfinding algorithms like Dijkstra's and A* can be implemented to perform dynamic pathfinding efficiently and effectively. The project source code represents the necessary data structures in the form of a basic unit that is held by a larger container object (i.e. Connection and graph). The objects are used to perform graph-based search methods which can calculate the most effective way to get from one "node" to another.

The nodes are quantized and localized using simple formulas that enable the pathfinding to be loosely coupled with the movement behaviors. The various parameters involved with constructing the game environment are shown such as adjusting corridor size, pathfinding algorithm, and graph size. Finally, the algorithms are implemented in a simple game using SFML and the boid is automatically routed to wherever the user clicks. These algorithms have a fairly low time complexity which can be further optimized using techniques such as a priority heap or storing other data to retain where a node has been open, closed, or visited.

ACKNOWLEDGMENT

Dr. Tiffany Barnes, thank you for instructing us in how to implement the various algorithms detailed in this report.

V. APPENDIX

```

Generating a graph!
Random graph generation:
The graph has 3000 vertices
and has 9000 edges.
Graph Analysis Running!
fromNode: 157 | toNode: 7 | Cost: 0.461195

fromNode: 1249 | toNode: 157 | Cost: 0.506753
fromNode: 2877 | toNode: 1249 | Cost: 3.48784
fromNode: 1 | toNode: 2877 | Cost: 5.89721

Graph Analysis Complete!
Dijkstra took 231 microseconds
fromNode: 157 | toNode: 7 | Cost: 0.461195

fromNode: 1249 | toNode: 157 | Cost: 0.506753
fromNode: 2877 | toNode: 1249 | Cost: 3.48784
fromNode: 1 | toNode: 2877 | Cost: 5.89721

Graph Analysis Complete!
A* with Heuristic took 762 microseconds
fromNode: 157 | toNode: 7 | Cost: 0.461195

fromNode: 1249 | toNode: 157 | Cost: 0.506753
fromNode: 2877 | toNode: 1249 | Cost: 3.48784
fromNode: 1 | toNode: 2877 | Cost: 5.89721

Graph Analysis Complete!
A* with Heuristic took 1081 microseconds

```

Fig. 2: Speed Comparison between algorithms on a random graph with 3000 nodes and 9000 edges starting a node 1 and ending at node 7.

```

fromNode: 245 | toNode: 285 | Cost: 1
fromNode: 205 | toNode: 245 | Cost: 1
fromNode: 204 | toNode: 205 | Cost: 1
fromNode: 203 | toNode: 204 | Cost: 1
fromNode: 202 | toNode: 203 | Cost: 1
fromNode: 162 | toNode: 202 | Cost: 1
fromNode: 122 | toNode: 162 | Cost: 1
fromNode: 82 | toNode: 122 | Cost: 1
fromNode: 42 | toNode: 82 | Cost: 1

A* with Euclidean took 1300 microseconds

```

Fig. 3: A* algorithm with Euclidean heuristic on SFML map starting a node 42 and ending at node 1055.

```

fromNode: 325 | toNode: 365 | Cost: 1
fromNode: 285 | toNode: 325 | Cost: 1
fromNode: 245 | toNode: 285 | Cost: 1
fromNode: 205 | toNode: 245 | Cost: 1
fromNode: 204 | toNode: 205 | Cost: 1
fromNode: 203 | toNode: 204 | Cost: 1
fromNode: 202 | toNode: 203 | Cost: 1
fromNode: 162 | toNode: 202 | Cost: 1
fromNode: 122 | toNode: 162 | Cost: 1
fromNode: 82 | toNode: 122 | Cost: 1
fromNode: 42 | toNode: 82 | Cost: 1
A* with Manhattan took 11395 microseconds

```

Fig. 4: A* algorithm with Manhattan heuristic on SFML map starting a node 42 and ending at node 1055.

```

fromNode: 325 | toNode: 365 | Cost: 1
fromNode: 285 | toNode: 325 | Cost: 1
fromNode: 245 | toNode: 285 | Cost: 1
fromNode: 205 | toNode: 245 | Cost: 1
fromNode: 204 | toNode: 205 | Cost: 1
fromNode: 203 | toNode: 204 | Cost: 1
fromNode: 202 | toNode: 203 | Cost: 1
fromNode: 162 | toNode: 202 | Cost: 1
fromNode: 122 | toNode: 162 | Cost: 1
fromNode: 82 | toNode: 122 | Cost: 1
fromNode: 42 | toNode: 82 | Cost: 1
Dijkstra took 7747 microseconds

```

Fig. 5: Dijkstra's algorithm on SFML map starting a node 42 and ending at node 1055.

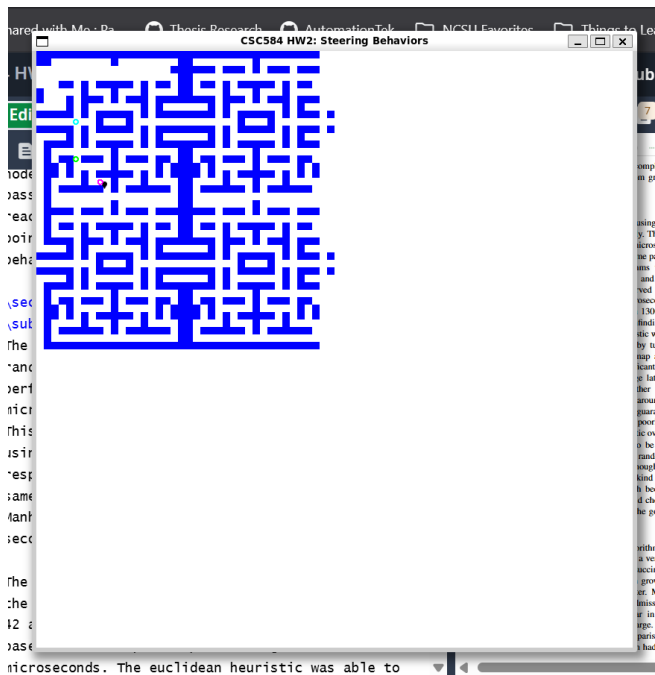


Fig. 6: Corridor set to size 10

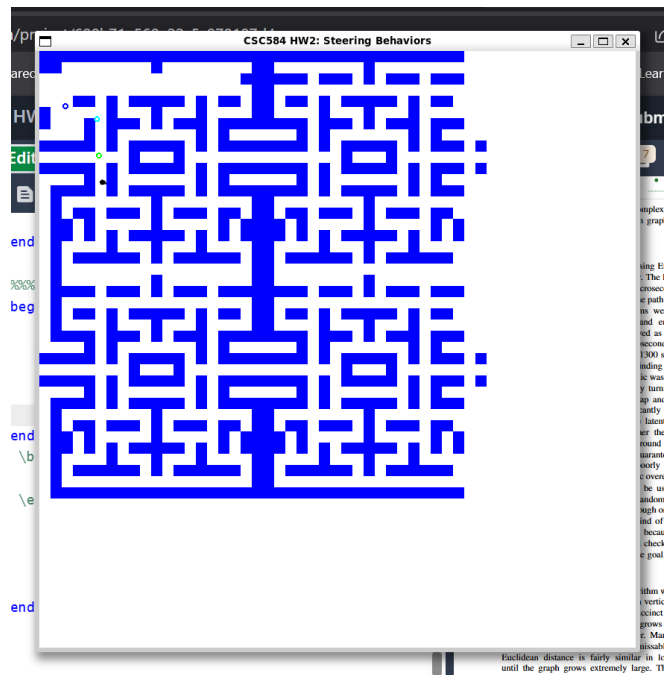


Fig. 7: Corridor set to size 15

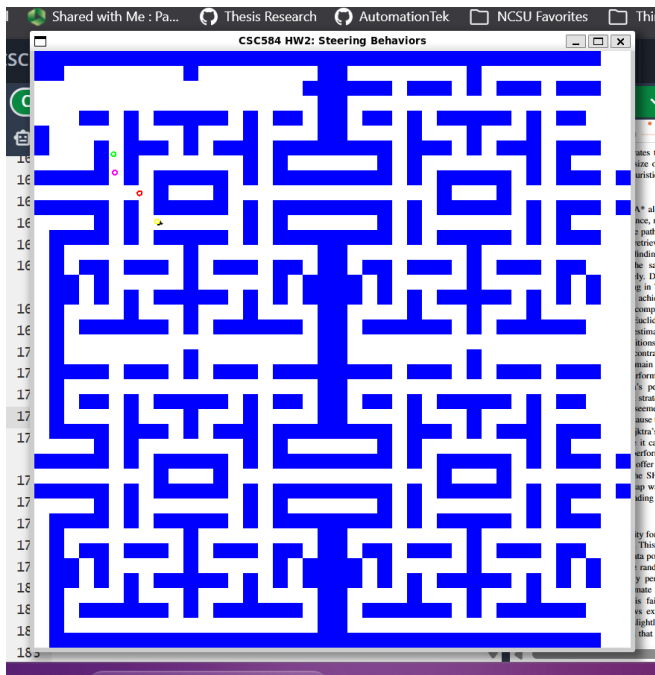


Fig. 8: Corridor set to size 20

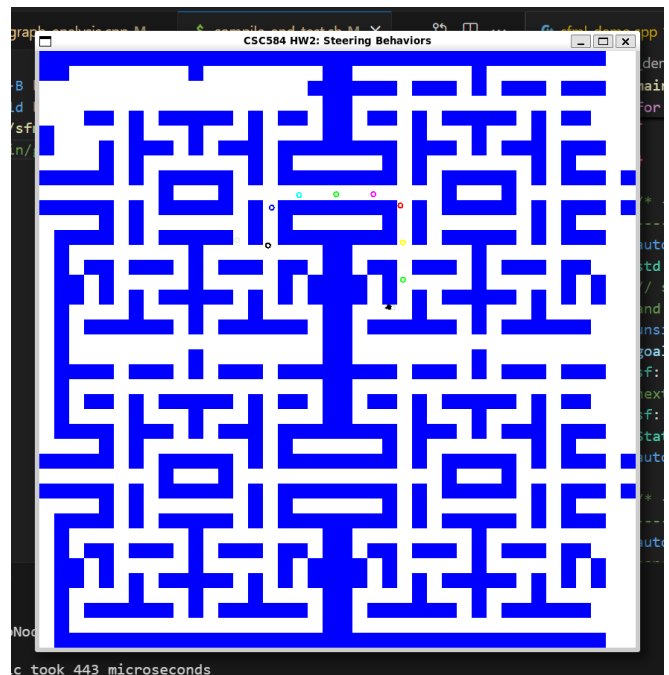


Fig. 9: Image of Euclidean pathfinding


```

sf::Vector2f calculatePositionfromNode(unsigned int node)
{
    // calculate the row and column index from the node index
    int row_index = node / MAP_WIDTH;
    int col_index = node % MAP_WIDTH;
    float x_pos = static_cast<float>(row_index * CELL_SIZE + CELL_SIZE / 2);
    float y_pos = static_cast<float>(col_index * CELL_SIZE + CELL_SIZE / 2);
    auto position = sf::Vector2f(y_pos, x_pos);
    return position;
}

```

Fig. 12: Code for calculating map position from node

```

unsigned int calculateNodeIndex(sf::Vector2f position)
{
    return (static_cast<unsigned int>(position.y) / CELL_SIZE) * MAP_WIDTH + static_cast<unsigned int>(position.x) / CELL_SIZE;
}

```

Fig. 13: Code for calculating node index from map position

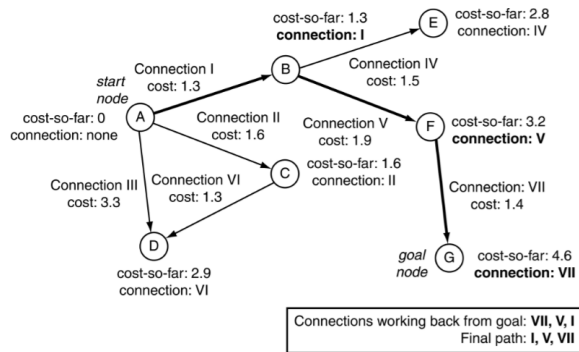


Fig. 14: small graph from AI for Game (3rd Ed.)